

CAKE DELIGHT

Cloud-Native Microservices Engineering (Stream Project)

Spring Boot • MongoDB • RabbitMQ • Docker • Kubernetes • Minikube

1. Project Overview

Cake Delight is a cloud-native microservices application designed around a complete cake-shopping customer journey. Users can browse and filter cakes, add products to a basket, modify basket contents, complete checkout, view orders, submit ratings and reviews, and receive an order-confirmation email.

The implementation uses four independently deployable Spring Boot microservices: Cake Catalog, Order, Rating, and Notification. MongoDB provides persistence, RabbitMQ provides asynchronous event communication, Docker provides containerization, and Kubernetes manages the services during the local cloud-native deployment through Minikube.

2. Project Objectives

- Implement business capabilities using a microservices architecture.
- Maintain clear service boundaries for catalog, order, rating, and notification capabilities.
- Expose REST APIs for synchronous application communication.
- Use RabbitMQ for asynchronous order-completion notification processing.
- Package services as Docker containers.
- Deploy and manage the application using Kubernetes.
- Provide database-backed persistence using MongoDB.
- Demonstrate an end-to-end customer workflow.

3. Technology Stack

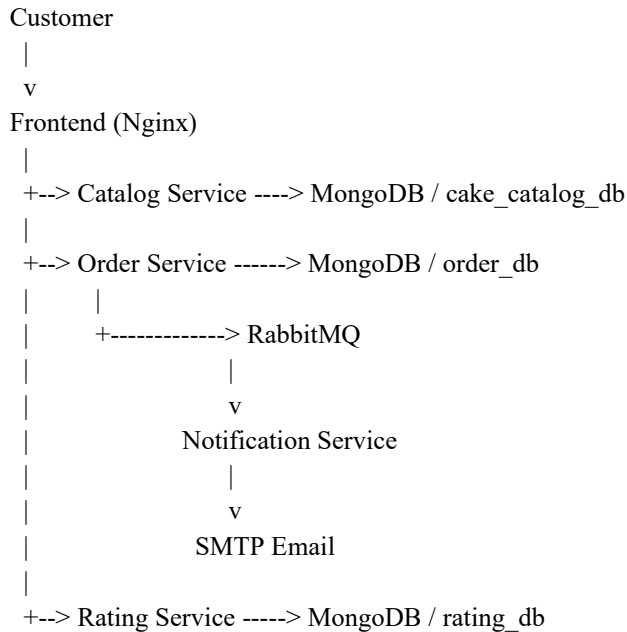
Component	Technology	Purpose
Backend	Spring Boot / Java 17	Independent microservices and REST APIs
Database	MongoDB 7	Catalog, basket/order and rating persistence
Messaging	RabbitMQ 3 Management	Order-completion event delivery
Frontend	HTML, CSS, JavaScript	Customer-facing user interface
Web Server	Nginx Alpine	Serves the frontend container
Containerization	Docker	Builds and runs application containers
Orchestration	Kubernetes / Minikube	Deployment, service discovery and workload management
Email	Spring Mail / Gmail SMTP	Order confirmation notification
Monitoring	Spring Boot Actuator	Basic health and information endpoints

4. Architecture

The system follows a microservices architecture in which each backend service owns a distinct business capability. Services are independently buildable and deployable. REST is used for application-facing synchronous operations, while RabbitMQ is used for the asynchronous order-completion notification flow.

There is no dedicated API Gateway in the current implementation. The frontend calls the required backend services directly. The architecture therefore focuses on the core microservices, persistence, messaging, containerization, and Kubernetes orchestration requirements.

5. Architecture Flow



6. Cake Catalog Microservice

The Cake Catalog Service maintains cake product information and exposes REST endpoints for retrieving, creating, updating, deleting, and searching cakes.

The Cake model contains cake ID, name, description, category, price, availability, and image URL. Bean validation ensures that the name and category are present and that the price is valid.

Method	Endpoint	Purpose
GET	/api/cakes	List all cakes
GET	/api/cakes/{cakeId}	Retrieve a cake by ID
POST	/api/cakes	Create a cake
PUT	/api/cakes/{cakeId}	Update a cake
DELETE	/api/cakes/{cakeId}	Delete a cake
GET	/api/cakes/search	Search/filter cakes

The search endpoint accepts optional name, category, minPrice, and maxPrice parameters. MongoDB stores the cakes in the catalog database.

7. Order Microservice

The Order Service manages customer baskets and order creation. A basket is associated with a customer email and contains cake ID, cake name, price, and quantity. Basket totals are recalculated when items are added, updated, or removed.

During checkout, the service validates that a basket exists and is not empty, creates the order, sets its status to CONFIRMED, persists it, publishes the order-completion event, and then removes the completed basket.

Method	Endpoint	Purpose
POST	/api/orders/cart?customerEmail={email}	Add an item to a basket
GET	/api/orders/cart/{customerEmail}	Retrieve a basket
PUT	/api/orders/cart/{customerEmail}/{cakeId}?quantity={quantity}	Update item quantity
DELETE	/api/orders/cart/{customerEmail}/{cakeId}	Remove an item
POST	/api/orders/checkout	Create an order from the basket
GET	/api/orders/{orderId}	Retrieve an order
GET	/api/orders/customer/{email}	Retrieve customer orders

8. Rating Microservice

The Rating Service allows customers to submit ratings and reviews for individual cakes. Ratings are validated between 1 and 5 and customer email addresses are validated.

Method	Endpoint	Purpose
POST	/api/ratings	Create a rating/review
GET	/api/ratings/cake/{cakeId}	Retrieve ratings for a cake
GET	/api/ratings/cake/{cakeId}/average	Calculate the average rating
GET	/api/ratings/{ratingId}	Retrieve a rating
DELETE	/api/ratings/{ratingId}	Delete a rating

The average-rating operation returns 0.0 when no ratings exist and rounds the calculated average to one decimal place.

9. Notification Microservice

The Notification Service is event-driven. It listens for order-completion events published by the Order Service through RabbitMQ. When an event is received, the service creates an email containing customer and order details and sends it through SMTP.

The email includes the customer name, order ID, cake names, quantities, prices, subtotals, total amount, and phone number.

SMTP credentials are supplied through environment variables rather than storing the password in the application properties file.

10. RabbitMQ Event Architecture

The Order Service publishes an OrderCompletedEvent after an order is successfully persisted. The event is sent through the direct exchange named order.exchange with routing key order.completed.

Element	Value
Exchange	order.exchange
Exchange type	Direct
Routing key	order.completed
Queue	order.notification.queue
Consumer	Notification Service

The event contract contains orderId, customerName, email, phoneNumber, totalAmount, and items. This creates loose coupling between order processing and notification delivery: the Order Service does not directly invoke the Notification Service to send the email.

11. Database Design

Service	Database	Primary Data
Catalog Service	cake_catalog_db	cakes
Order Service	order_db	baskets and orders
Rating Service	rating_db	ratings
Notification Service	No application database shown	Notification processing is event-driven

The Kubernetes manifests use the MongoDB service name mongodb for internal connectivity. Separate logical MongoDB databases are used for the catalog, order, and rating capabilities.

12. Frontend

The frontend is a lightweight HTML, CSS, and JavaScript application served through Nginx. Its JavaScript modules handle catalog retrieval, filtering, basket operations, checkout, customer orders, and rating interactions.

The basket is maintained in browser localStorage for the customer-facing basket experience. During checkout, basket contents are synchronized with the Order Service before the checkout operation is submitted.

13. Docker Containerization

Each Spring Boot service has its own Dockerfile based on Eclipse Temurin JDK 17. The Dockerfiles copy the Maven-built application JAR into the image and expose the service's application port.

The frontend uses the nginx:alpine image and serves the static application on port 80.

The JAR files in Maven target directories are generated build artifacts and are not required in source control. They can be regenerated before Docker image creation.

14. Kubernetes Deployment

The k8s directory contains Deployment and Service definitions for MongoDB, RabbitMQ, Catalog, Order, Rating, Notification, and Frontend.

Workload	Service Type	Port	Role
mongodb	ClusterIP	27017	Database
rabbitmq	ClusterIP	5672 / 15672	Messaging and management
catalog	ClusterIP	8081	Internal catalog API
order	ClusterIP	8082	Internal order API
rating	ClusterIP	8083	Internal rating API
notification	ClusterIP	8084	Internal notification service
frontend	NodePort	80	Browser-accessible frontend

The backend services use Kubernetes Service names for service discovery. For example, the Catalog Service connects to mongodb:27017 and the Order and Notification Services connect to rabbitmq:5672.

The application images are loaded into Minikube and the manifests use imagePullPolicy: Never, allowing the local images to be used without a public container registry.

15. Kubernetes Secret and Configuration

The Notification Service reads MAIL_USERNAME and MAIL_PASSWORD from environment variables. In Kubernetes, these values are supplied from the mail-secret Secret using secretKeyRef entries named username and password.

This prevents the mail password from being written directly into the Kubernetes Deployment manifest.

16. Health, Error Handling and Maintainability

The services expose Spring Boot Actuator health and information endpoints through the configured management endpoints. The Catalog, Order, and Rating services also contain service-specific exception classes and global exception handlers.

The Order Service explicitly handles empty or missing baskets and missing orders. The Catalog and Rating services provide not-found exception handling. Kubernetes pod status, logs, descriptions, and events provide basic operational diagnostics.

17. End-to-End Customer Journey

- Customer opens the Cake Delight frontend.
- Frontend loads cakes from the Catalog Service.
- Customer filters cakes by name, category, or price range.
- Customer adds available cakes to the basket and changes quantities when required.
- Checkout information is collected and basket items are synchronized with the Order Service.
- Order Service creates and persists the order with CONFIRMED status.
- Order Service publishes the OrderCompletedEvent to RabbitMQ.
- RabbitMQ routes the event to the notification queue.
- Notification Service consumes the event and sends the confirmation email.
- Customer can view previous orders and submit ratings/reviews for cakes.

18. Setup and Execution

18.1 Docker Compose

```
docker compose up --build
```

The Docker Compose deployment runs the frontend, four backend services, MongoDB, and RabbitMQ together on a Docker network.

18.2 Minikube and Kubernetes

```
minikube start --driver=docker
kubectl get nodes
kubectl apply -f .\k8s\mongodb.yaml
kubectl apply -f .\k8s\rabbitmq.yaml
kubectl apply -f .\k8s\catalog.yaml
kubectl apply -f .\k8s\order.yaml
kubectl apply -f .\k8s\rating.yaml
kubectl apply -f .\k8s\notification.yaml
kubectl apply -f .\k8s\frontend.yaml
kubectl get pods
kubectl get services
```

The mail-secret must exist before the Notification Service pod can start successfully. The frontend can be opened with minikube service frontend --url.

19. API and Communication Summary

Communication	Mechanism	Example
Frontend to Catalog	REST	GET /api/cakes
Frontend to Order	REST	POST /api/orders/checkout
Frontend to Rating	REST	POST /api/ratings
Order to RabbitMQ	AMQP event	order.exchange / order.completed
RabbitMQ to Notification	AMQP listener	order.notification.queue
Notification to mail server	SMTP	Gmail SMTP

20. Requirement Compliance

Assessment Requirement	Implementation Status	Evidence
Microservices architecture	Implemented	Four independently deployable Spring Boot services
Catalog capability	Implemented	Catalog CRUD and filtering APIs
Basket management	Implemented	Add, retrieve, update and remove basket operations
Checkout and orders	Implemented	Checkout creates persisted CONFIRMED orders
Ratings and reviews	Implemented	Rating CRUD/read operations and average calculation
Order completion event	Implemented	OrderCompletedEvent published through RabbitMQ
Event-driven notification	Implemented	Notification Service consumes RabbitMQ event
Email notification	Implemented	SMTP confirmation email
Database persistence	Implemented	MongoDB databases for catalog, order and rating
Docker containerization	Implemented	Dockerfiles for backend services and frontend
Kubernetes orchestration	Implemented	Deployment and Service manifests

Kubernetes service discovery	Implemented	ClusterIP Services and service-name based connections
Secrets management	Implemented	Kubernetes mail-secret
Health/monitoring	Basic	Spring Boot Actuator and Kubernetes diagnostics
API Gateway	Not implemented	No dedicated gateway component in current solution
Public cloud deployment	Not implemented	Application is demonstrated locally through Minikube
Production-grade persistent Kubernetes storage	Not implemented	MongoDB currently uses emptyDir in the supplied manifest
Advanced observability	Basic	Actuator, logs and Kubernetes events

21. Known Limitations

- The current demonstration uses Minikube locally rather than a public cloud Kubernetes service.
- There is no dedicated API Gateway in the current implementation.
- MongoDB uses emptyDir storage in the Kubernetes manifest, so database data is not protected by a production-grade persistent volume.
- The Kubernetes manifests use single replicas for the current demonstration.
- Observability is basic and is based on Actuator, application logs, and Kubernetes diagnostics rather than a full metrics/logging stack.

These limitations do not change the implemented core microservices, REST, messaging, Docker, database, and Kubernetes capabilities. They identify areas that would be appropriate for a production deployment.

22. Project Deliverables

- Source code for Catalog, Order, Rating, and Notification microservices.
- Frontend source code.
- Dockerfiles for application components.
- Docker Compose configuration.
- Kubernetes Deployment and Service manifests.
- MongoDB and RabbitMQ deployment definitions.
- Kubernetes Secret configuration reference for mail credentials.
- API endpoints implemented by the services.
- RabbitMQ order-completion event contract.
- README with setup and execution instructions.

23. Submission and Repository Hygiene

Generated Maven build output is excluded from source control using .gitignore. The .env file containing local mail credentials must also remain outside the Git repository. The source repository should contain the Java source, frontend files, Maven configuration, Dockerfiles, Docker Compose configuration, Kubernetes manifests, README, and project documentation.

Before submission, the repository should be cloned or downloaded from GitHub and its ZIP size should be checked against the assessment's 100 MB limit.

24. Conclusion

Cake Delight demonstrates a practical cloud-native microservices architecture with independently deployable services, REST APIs, MongoDB persistence, Docker containerization, Kubernetes orchestration, Kubernetes service discovery, RabbitMQ event-driven communication, and asynchronous email notification. The end-to-end workflow connects catalog browsing, basket management, checkout, order persistence, event publication, notification processing, and customer ratings into a single application.

The implementation is suitable for demonstrating the core requirements of the Cloud Native Microservices Engineering capstone while clearly identifying the differences between the local demonstration environment and a production-grade public cloud deployment.